



Advanced Linux & Shell Scripting



Today's Lesson

Vim Editor

Shell Scripting & Automation

Text processing command

Linux Cronjob





Vim Editor

Text Editor

Ubuntu Server does not provide a graphical user interface, so you will use **command-line editors** like to edit text files.

These are lightweight and available even on servers without a GUI.

- **Vi / Vim** → The classic, powerful modal editor (pre-installed on most Linux distros).
- **Nano** → Beginner-friendly, simple editor with on-screen shortcuts.
- **Emacs** → Very powerful, extensible editor (more than just text editing, can be like an IDE).
- **Neovim** → Modern, improved fork of Vim with extra features.
- ... Many more ...

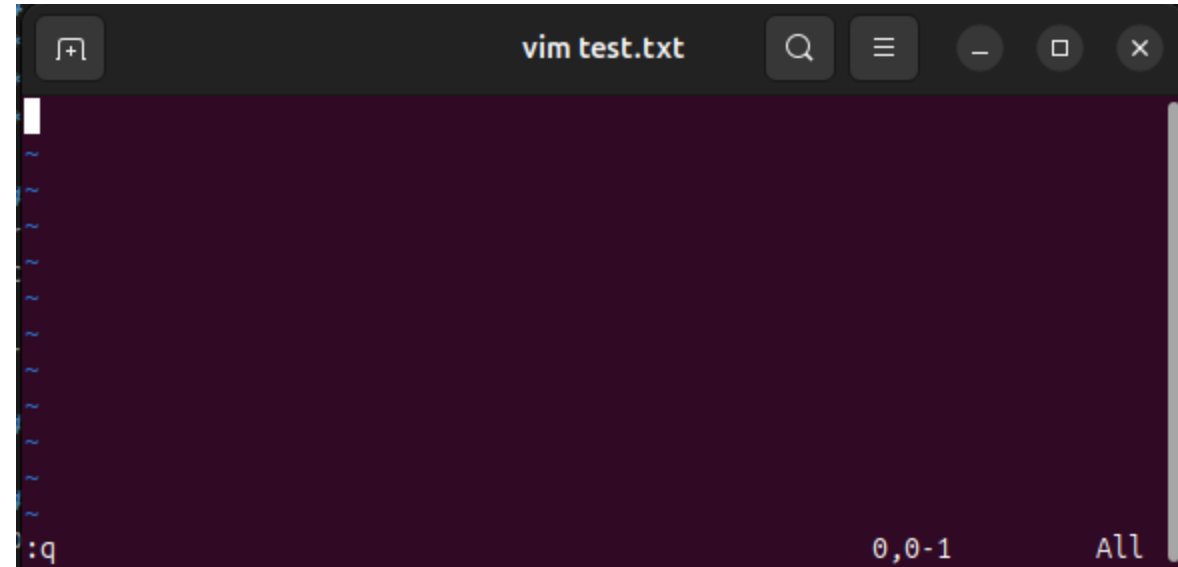
Vim Editor

What is Vim?

- **Vim** = *Vi IMproved* – an enhanced version of the original Unix editor **vi**.
- Available by default in most Linux/Unix systems.
- Commonly used for editing **configuration files, scripts, code**, etc.
- Lightweight, modal editor (different modes for editing vs. commands).

How to Open Vim

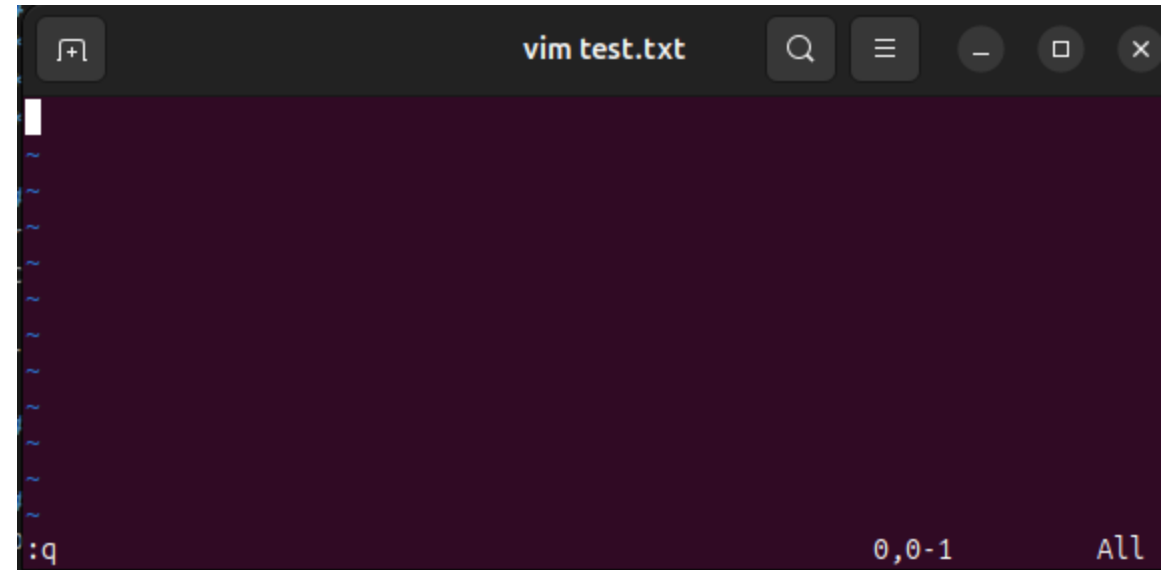
- `vim filename.txt` # Open (or create) a file



Modes in Vim

This is the most important concept!

- **Normal Mode** (default when Vim opens)
 - Used for navigation, copy/paste, delete, etc.
 - Press **Esc** anytime to return.
- **Insert Mode** (typing mode)
 - Enter by pressing **i**, **a**, or **o**.
 - Example: press **i** → type text → press **Esc** to go back to normal.
- **Command-Line Mode** (for saving, quitting, searching)
 - Enter by pressing **:** in normal mode.
 - Example: **:w** (save), **:q** (quit).



Vim cheat-sheet

Mode	Key(s)	Action
Insert Mode	i, a, o	Start entering text
Save	:w	Save file
Quit	:q	Quit
Save + Quit	:wq	Save and exit
Quit w/o Save	:q!	Exit discard changes
Delete line	dd	Cut/delete line
Copy line	yy	Copy line
Paste	p	Paste after cursor
Undo / Redo	u / Ctrl+r	Undo & redo changes
Search	/word	Find word
Help	:help	Open help menu



Linux Shell scripting

Introduction to Shell Scripting

What is Shell Scripting?

- A shell script is a file containing a sequence of commands that are executed by the Linux shell.
- Useful for automation of repetitive tasks, system administration, and custom workflows.

Why Shell Scripting?

- Reduces manual effort
- Automates repetitive tasks
- Helps create system maintenance workflows

```
1 #!/bin/bash
2 echo "System Health Report"
3 echo "-----"
4 echo "Disk Usage:"
5 df -h | grep '/dev/'
6 echo
7 echo "Memory Usage:"
8 free -h
9 echo
10 echo "Top 5 Processes consuming CPU:"
11 ps -eo pid,comm,%cpu --sort=-%cpu | head -n 6
```

Script file structure

```
1 #!/bin/bash
2 echo "System Health Report"
3 echo "-----"
4 echo "Disk Usage:"
5 df -h | grep '/dev/'
6 echo
7 echo "Memory Usage:"
8 free -h
9 echo
10 echo "Top 5 Processes consuming CPU:"
11 ps -eo pid,comm,%cpu --sort=-%cpu | head -n 6
```

shebang

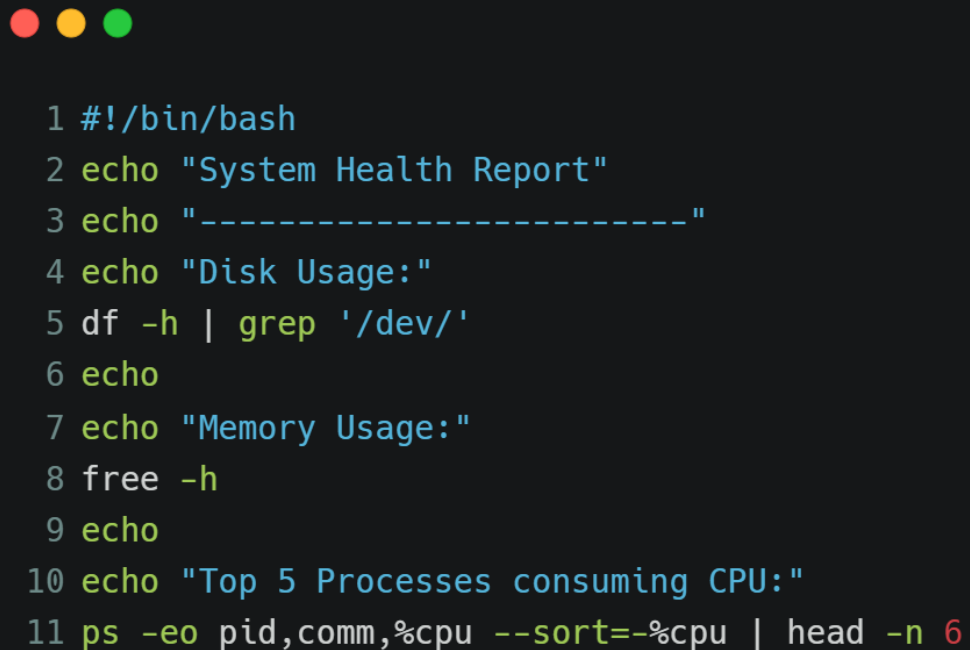
The **shebang** (also called **hashbang**) is a character sequence at the very beginning of a script file

`#!/bin/bash`

- Here `#!/bin/bash` means that the **Bash shell** will interpret and run the commands in the script.
- If you run `./myscript.sh`, Linux looks at the shebang line, and runs `/bin/bash myscript.sh`.

The rest file content is script logic

Execute script file



```
1 #!/bin/bash
2 echo "System Health Report"
3 echo "-----"
4 echo "Disk Usage:"
5 df -h | grep '/dev/'
6 echo
7 echo "Memory Usage:"
8 free -h
9 echo
10 echo "Top 5 Processes consuming CPU:"
11 ps -eo pid,comm,%cpu --sort=-%cpu | head -n 6
```

- Save file as `script.sh`
- Make executable: `chmod +x script.sh`
- Run: `./script.sh`



Advance script file

- Like a programming language, script file also have some “feature” as other programming language such as
 - Variables and Parameter Handling
 - Conditional Structures
 - Loop
 - Functions and Modular Programming

Variables

- In shell scripts, variables are used to store data (string, number, file path, etc).
- No data type declarations; everything is treated as a string.

```
1 #!/bin/bash
2 name="Alice"
3 age=25
4
5 echo "Name: $name"
6 echo "Age: $age"
```

Parameter Handling (User input)

Script also can take input parameter:

```
1 #!/bin/bash
2 echo "Script Name: $0"
3 echo "First argument: $1"
4 echo "Second argument: $2"
5 echo "Total arguments: $#"
```

Execute: `./script.sh apple banana`
Output:

```
1 Script Name: ./myscript.sh
2 First argument: apple
3 Second argument: banana
4 Total arguments: 2
```

Conditional (If / Else)

If / elif / else

```
1 if [ $num -gt 80 ]; then
2     echo "High"
3 elif [ $num -gt 50 ]; then
4     echo "Medium"
5 else
6     echo "Low"
7 fi
```

case

```
1 #!/bin/bash
2 echo "Enter choice (start/stop/restart):"
3
4 case $1 in # Read from argument
5     start) echo "Service starting..." ;;
6     stop) echo "Service stopping..." ;;
7     restart) echo "Service restarting..." ;;
8     *) echo "Invalid option" ;;
9 esac
```

Loops

For loop:

```
1 for i in 1 2 3 4 5
2 do
3     echo "Iteration $i"
4 done
```

While loop:

```
1 count=1
2 while [ $count -le 5 ]
3 do
4     echo "Count = $count"
5     count=$((count+1))
6 done
```

Functions and Modular Programming

Functions let us reuse code and make scripts modular.

```
1 #!/bin/bash
2 greet() {
3     echo "Hello, $1!"
4 }
5 add_numbers() {
6     sum=$(( $1 + $2 ))
7     echo "Sum: $sum"
8 }
9
10 # Call functions
11 greet "Alice"
12 add_numbers 10 20
```

Output:

Hello, Alice!
Sum: 30



Text Processing Commands

Common Commands

Command	Usage Example	What it Does
cat	<code>cat file.txt</code>	Show contents of file
head	<code>head -n 5 file.txt</code>	First 5 lines
tail	<code>tail -f server.log</code>	Follow live logs
grep	<code>grep "error" logfile.txt</code>	Search lines by text
cut	<code>cut -d ":" -f1 /etc/passwd</code>	Extract usernames
sort	<code>sort data.txt</code>	Sort lines
uniq	<code>uniq names.txt</code>	Remove duplicates
awk	<code>awk '{print \$1, \$3}' file.txt</code>	Print specific columns
sed	<code>sed 's/foo/bar/g' file.txt</code>	Replace text

cat

Purpose:

- Display the content of files
- Concatenate multiple files

Syntax:

- `cat file.txt`
- `cat file1.txt file2.txt > combined.txt`

Examples:

- `cat /etc/passwd` # Show user account info
- `cat -n notes.txt` # Show file with line numbers
- `cat file1.txt file2.txt` # Display two files in sequence
- `cat file1.txt file2.txt > merged.txt` # Merge into new file

head / tail

Purpose:

- Show the Beginning or End of Files
- head → first lines
tail → last lines

Syntax & Examples:

- head -n 10 logfile.txt # show first 10 lines
- tail -n 20 logfile.txt # show last 20 lines
- tail -f /var/log/syslog # continuously monitor (real-time logs)

grep

Purpose:

- Search text using keywords or regular expressions.

Syntax & Examples:

- `grep "error" logfile.txt` # Find lines containing the word 'error'
- `grep -i "Warning" logfile.txt` # Case insensitive search
- `grep -r "main()" src/` # Search recursively in directory
- `grep -n "linux" notes.txt` # Show line numbers with matches

cut

Purpose:

- Extract specific columns from structured text (like CSV, TSV, or : delimited files).

Syntax & Examples:

- `cut -d ":" -f1 /etc/passwd` # Print column 1 (usernames)
- `cut -d "," -f2,4 data.csv` # Get 2nd & 4th columns from CSV

sort & uniq

Purpose:

- Sorting and Removing Duplicates

Syntax & Examples:

- `sort names.txt` # Sort alphabetically
- `sort -n numbers.txt` # Numeric sort
- `sort -r names.txt` # Reverse sort
- `sort names.txt | uniq` # Remove duplicates
- `sort names.txt | uniq -c` # Count duplicate occurrences

awk

Purpose:

- Process text **column by column** and apply conditions.

Syntax & Examples:

- `awk '{print $1}' data.txt` # Print first column
- `awk '{print $1, $3}' data.txt` # Print first and third column
- `awk '$3 > 80 {print $1, $3}' marks.txt` # Print names where 3rd column > 80
- `awk -F, '{print $1}' data.csv` # Use comma as delimiter

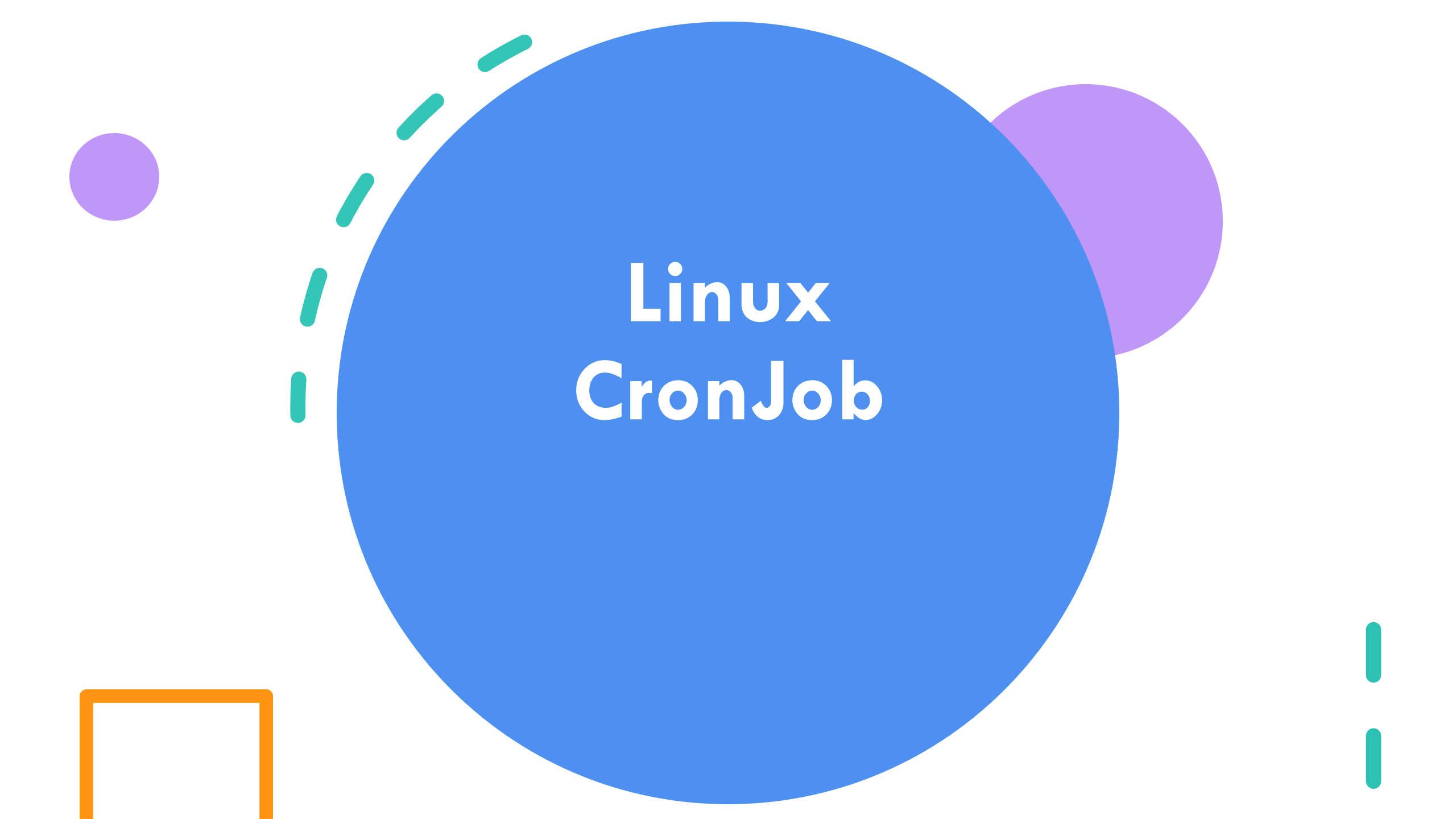
sed – Stream Editor (Search and Replace)

Purpose:

- Edit or transform text stream on the fly.\

Syntax & Examples:

- `sed 's/error/issue/' logfile.txt` # Replace first occurrence per line
- `sed 's/error/issue/g' logfile.txt` # Replace all occurrences (global)
- `sed -n '5,10p' file.txt` # Print only lines 5-10
- `sed '/^#/d' config.conf` # Delete all lines starting with #



Linux CronJob

What is Linux CronJob

Cron is a **time-based job scheduling** daemon found in Unix-like operating systems, including Linux distributions. Cron runs in the background and operations scheduled with cron, referred to as "cron jobs," are executed automatically.

Example:

- You want to backup sql database for every 5 minutes
- You want to backup and delete log file everyday
- You want to restart machine every Sunday at 3:00 AM

=> Cronjob is the best for that kind tasks

Install CronJob

Almost every Linux distribution has some form of cron installed by default

Check if cron is already installed:

- `systemctl status cron`

If not installed:

- `sudo apt update`
- `sudo apt install cron`
- `sudo systemctl enable cron`

Using CronJob

Ubuntu support `crontab` command to interact with conjob seting

- `crontab -e` → Edit cron jobs
- `crontab -l` → List user cron jobs

Cron Schedule

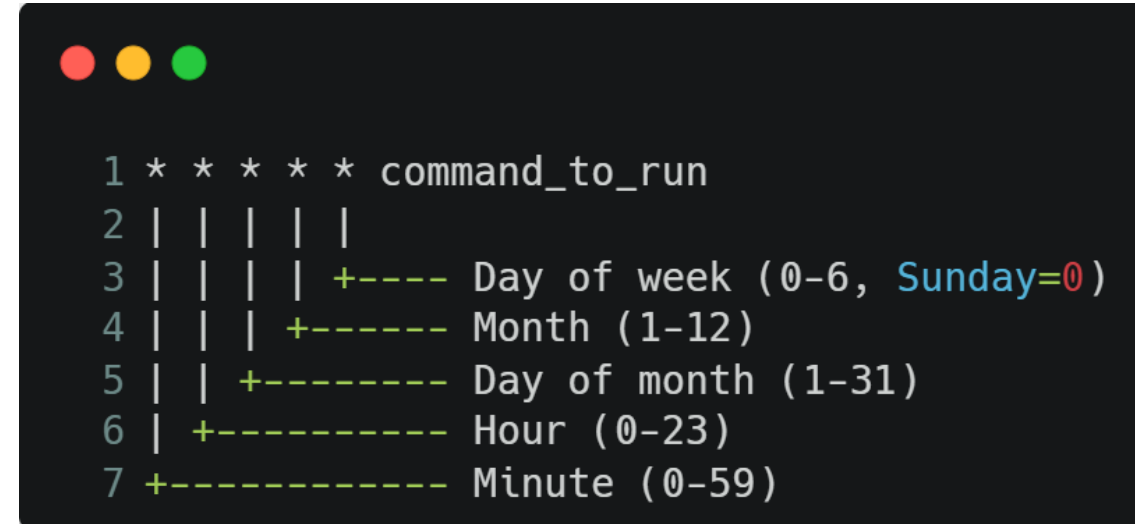


```
1 * * * * * command_to_run
2 | | | | |
3 | | | | +----- Day of week (0-6, Sunday=0)
4 | | | +----- Month (1-12)
5 | | +----- Day of month (1-31)
6 | +----- Hour (0-23)
7 +----- Minute (0-59)
```

Using CronJob

Example:

- Run backup script every day at 2 AM
 - `0 2 * * * /home/user/backup.sh`
- Clear temp folder every Sunday at midnight
 - `0 0 * * 0 rm -rf /tmp/*`
- ProTips: Visit <https://crontab.guru/> for more schedule example



```
1 * * * * * command_to_run
2 | | | | |
3 | | | | +---- Day of week (0-6, Sunday=0)
4 | | | +----- Month (1-12)
5 | | +----- Day of month (1-31)
6 | +----- Hour (0-23)
7 +----- Minute (0-59)
```



Practice Lab